Problem Formulation: Reinforcement Learning for Humanoid Robot Locomotion

1 Introduction

This document presents the formal mathematical formulation of the problem addressed in this thesis: the development of a reinforcement learning framework for goal-directed locomotion of the Unitree G1 humanoid robot using a digital twin implemented in Isaac Lab.

2 Problem Statement

We consider the problem of training a control policy for a humanoid robot to navigate towards randomly sampled goal positions while maintaining stability and executing smooth movements. The policy is learned through reinforcement learning in a simulated environment and is designed to facilitate transfer to the real robotic platform.

3 Markov Decision Process Formulation

The locomotion problem is modeled as a finite-horizon Markov Decision Process (MDP) defined by the tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \rho_0, \gamma, T \rangle$.

3.1 State Space

The state space $\mathcal{S} \subseteq \mathbb{R}^n$ represents the complete information available to the learning agent at each timestep. The state at time t is denoted as $s_t \in \mathcal{S}$.

3.1.1 State Representation

The state s_t is composed of the following components:

$$s_t = [\mathbf{v}_t, \mathbf{q}_t, \dot{\mathbf{q}}_t, \mathbf{g}_t, \mathbf{a}_t]^T \in \mathbb{R}^{n_s} \tag{1}$$

where:

- $\mathbf{v}_t = [v_x^t, v_y^t, \omega_z^t]^T \in \mathbb{R}^3$ is the linear and angular velocity of the robot's base expressed in the base frame:
$$\mathbf{v}_t = \left[\dot{x}^t, \dot{y}^t, \dot{\theta}_z^t\right]^T \tag{2}$$
- $\mathbf{q}_t = [q_1^t, q_2^t, \ldots, q_{29}^t]^T \in \mathbb{R}^{29}$ is the vector of joint positions for all 29 degrees of freedom of the Unitree G1 robot, constrained by:
$$q_i^{\min} \leq q_i^t \leq q_i^{\max} \quad \forall i \in \{1, 2, \ldots, 29\} \tag{3}$$
- $\dot{\mathbf{q}}_t = [\dot{q}_1^t, \dot{q}_2^t, \ldots, \dot{q}_{29}^t]^T \in \mathbb{R}^{29}$ is the vector of joint velocities
- $\mathbf{g}_t = [g_x^t, g_y^t]^T \in \mathbb{R}^2$ is the goal position relative to the robot's base expressed in the base frame (base-centric coordinates)
- $\mathbf{a}_t = [\ddot{x}_t, \ddot{y}_t, \ddot{z}_t, \omega_x^t, \omega_y^t, \omega_z^t]^T \in \mathbb{R}^6$ is the linear acceleration and angular velocity measured by the inertial measurement unit (IMU)

3.1.2 Observation Space

The observation space $\mathcal{O} \subseteq \mathbb{R}^{n_o}$ contains the observations available to the policy. Due to computational limitations and the need for temporal context in neural network training, observations include a history of past states:

$$o_t = [s_t, s_{t-1}, \ldots, s_{t-h+1}]^T \in \mathbb{R}^{n_o} \tag{4}$$

where h is the history length. The total observation dimension is:

$$n_o = h \times n_s = 8 \times (3 + 29 + 29 + 2 + 6) = 8 \times 69 = 552 \text{ (with history)} \tag{5}$$

However, in practice, privileged information compression is applied during training, resulting in:

$$n_o = 125 \text{ (effective observation dimension)} \tag{6}$$

3.2 Action Space

The action space $\mathcal{A} \subseteq \mathbb{R}^m$ defines the set of possible actions the agent can take. The action at time t is denoted as $a_t \in \mathcal{A}$.

3.2.1 Action Representation

The action is defined as a vector of desired joint positions:

$$a_t = [q_d^{1,t}, q_d^{2,t}, \ldots, q_d^{29,t}]^T \in [-\pi, \pi]^{29} \tag{7}$$

where $q_d^{i,t}$ is the desired target angle for joint i at timestep t.

3.2.2 Motor Control

The desired joint angles are converted to motor commands through a proportional-derivative (PD) controller:

$$\tau_i^t = K_p^i \left(q_d^{i,t} - q_i^t\right) + K_d^i \left(\dot{q}_d^{i,t} - \dot{q}_i^t\right) \tag{8}$$

where:

- τ_i^t is the torque command for joint i
- K_p^i is the proportional gain (set to 20.0 for all joints)
- K_d^i is the derivative gain (set to 0.5 for all joints)

- $\dot{q}_d^{i,t}$ is the desired joint velocity, computed from the action history

The motor torques are subject to physical constraints:

$$|\tau_i^t| \leq \tau_i^{\max} \quad \forall i \in \{1, 2, \ldots, 29\} \tag{9}$$

3.3 Transition Dynamics

The state transition follows the deterministic physics simulation provided by Isaac Lab:

$$s_{t+1} = f(s_t, a_t) \tag{10}$$

where f is the forward dynamics function implemented in the Isaac Sim physics engine. The transition is deterministic in the sense that given identical initial conditions and actions, the resulting state is identical. However, in practice, we treat it as stochastic due to the introduction of action noise during training.

3.4 Reward Function

The reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ provides learning signal to the agent:

$$r_t = R(s_t, a_t, s_{t+1}) \tag{11}$$

The total reward at each timestep is a weighted combination of multiple reward terms:

$$r_t = w_v r_v^t + w_d r_d^t + w_a r_a^t + w_s r_s^t + r_{\text{term}}^t \tag{12}$$

3.4.1 Velocity Tracking Reward

This reward encourages the robot to track a reference velocity:

$$r_v^t = \exp\left(-\frac{(||\mathbf{v}_t|| - v_{\text{ref}})^2}{2\sigma_v^2}\right) \tag{13}$$

where:

- $||\mathbf{v}_t|| = \sqrt{(v_x^t)^2 + (v_y^t)^2 + (\omega_z^t)^2}$ is the combined velocity magnitude

- $v_{\text{ref}} \in \{0.0, 0.5, 1.0, 1.5\}$ m/s is the reference velocity command
- $\sigma_v = 0.2$ is the standard deviation of the Gaussian reward

3.4.2 Distance to Goal Reward

This reward encourages the robot to move towards the goal:

$$r_d^t = \exp\left(-\frac{||\mathbf{g}_t||^2}{\sigma_d^2}\right) \tag{14}$$

where:

- $||\mathbf{g}_t|| = \sqrt{(g_x^t)^2 + (g_y^t)^2}$ is the distance to the goal
- $\sigma_d = 2.0$ controls the reward spread

3.4.3 Heading Alignment Reward

This reward encourages the robot to align its heading with the goal direction:

$$r_a^t = \cos(\theta_{\text{error}}^t) \tag{15}$$

where θ_{error}^t is the angular difference between the robot's heading and the direction to the goal:

$$\theta_{\text{error}}^t = \arctan 2(g_y^t, g_x^t) - \psi_t \tag{16}$$

with ψ_t being the robot's yaw angle in the world frame.

3.4.4 Action Smoothness Reward

This reward penalizes large changes in joint commands to promote smooth movements:

$$r_s^t = -\alpha \, ||a_t - a_{t-1}||_2^2 \tag{17}$$

where $\alpha = 0.01$ is the smoothness penalty weight.

3.4.5 Termination Reward

A bonus reward is given when the robot successfully reaches the goal:

$$r^t_{\text{term}} = \begin{cases} R_{\text{success}} = 5.0 & \text{if } ||\mathbf{g}_t|| < \epsilon_g \\ 0 & \text{otherwise} \end{cases} \tag{18}$$

where $\epsilon_g = 0.2$ m is the goal tolerance.

3.4.6 Reward Weights

The relative importance of each reward component is controlled by weight parameters:

$$(w_v, w_d, w_a, w_s) = (1.0, 1.0, 0.5, 0.01) \tag{19}$$

3.5 Episode Termination

An episode terminates when any of the following conditions is met:

$$\text{terminal}(s_t) = \begin{cases} \text{True} & \text{if } t \geq T \\ \text{True} & \text{if } ||\mathbf{g}_t|| < \epsilon_g \text{ (goal reached)} \\ \text{True} & \text{if } h_{\text{CoM}} < h_{\min} \text{ (fall)} \\ \text{False} & \text{otherwise} \end{cases} \tag{20}$$

where:

* $T = 500$ is the maximum episode length (50 seconds at 10 Hz control frequency)
* $h_{\min} = 0.3$ m is the minimum center-of-mass height threshold

3.6 Initial State Distribution

The initial state is sampled from a distribution ρ_0:

$$s_0 \sim \rho_0 \tag{21}$$

In this work, the initial state is determined by:

* Base position: reset to a fixed location
* Joint configuration: reset to a nominal standing pose
* Joint velocities: initialized to zero
* Goal position: sampled uniformly from a circular region

The goal position is sampled according to:

$$\mathbf{g}_0 \sim \text{Uniform}(\{(x, y) : 1.0 \leq \sqrt{x^2 + y^2} \leq 3.0\}) \tag{22}$$

3.7 Discount Factor and Horizon

The discount factor is set to:

$$\gamma = 0.99 \tag{23}$$

This implies that rewards 10 timesteps in the future are weighted approximately 90% of their immediate importance. The finite horizon $T = 500$ ensures that episodes have bounded length.

4 Policy Learning Problem

4.1 Trajectory Return

For a trajectory $(s_0, a_0, r_0, \ldots, s_T, a_T, r_T)$, the cumulative discounted return starting from timestep t is:

$$G_t = \sum_{\tau=t}^{T} \gamma^{\tau-t} r_\tau \tag{24}$$

4.2 Policy and Value Functions

The policy is represented as a stochastic mapping:

$$\pi_\theta : \mathcal{S} \to \mathbb{P}(\mathcal{A}) \tag{25}$$

In this work, the policy is parameterized as a Gaussian distribution:

$$\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \Sigma_\theta(s)) \tag{26}$$

where:

* $\mu_\theta(s) \in \mathbb{R}^{29}$ is the mean action, computed by a neural network with parameters θ
* $\Sigma_\theta(s) = \text{diag}(\sigma_1^2, \ldots, \sigma_{29}^2)$ is the diagonal covariance matrix
* The standard deviations σ_i are learned as part of training

The state-value function is defined as:

$$V_\phi(s) = \mathbb{E}_{a \sim \pi_\theta(\cdot|s)} \left[G_t \mid s_t = s \right] \tag{27}$$

which estimates the expected return from state s following policy π_θ.

4.3 Optimization Objective

The goal is to find the policy parameters θ^* that maximize the expected return:

$$\theta^* = \arg\max_\theta \mathbb{E}_{s \sim \rho_0} \left[G_0 \right] \tag{28}$$

This is equivalent to maximizing:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \gamma^t r_t \right] \tag{29}$$

where τ denotes a trajectory sampled from the policy π_θ.

5 Implementation Details

5.1 Neural Network Architecture

Both the policy and value function are implemented as multilayer perceptrons (MLPs):

$$\mu_\theta(s) = W_2^T \sigma(W_1^T s + b_1) + b_2 \tag{30}$$

where:

* Input dimension: $n_o = 125$ (observation size)
* Hidden layer dimension: $d_h = 256$
* Output dimension: $n_a = 29$ (action size)
* Activation function: $\sigma(\cdot) = \max(0, \cdot)$ (ReLU)
* Weights: $W_1 \in \mathbb{R}^{125\times256}$, $W_2 \in \mathbb{R}^{256\times29}$
* Biases: $b_1 \in \mathbb{R}^{256}$, $b_2 \in \mathbb{R}^{29}$

The value function follows a similar architecture with output dimension 1.

5.2 Training Algorithm

The policy is trained using Proximal Policy Optimization (PPO), a state-of-the-art on-policy reinforcement learning algorithm. The training procedure iteratively collects rollouts and updates the policy to maximize a surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t) \right] \tag{31}$$

where:

* $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio

- $\hat{A}_t$ is the estimated advantage function
- $\epsilon = 0.2$ is the clipping parameter

6 Summary of Hyperparameters

Table **??** summarizes the key hyperparameters used in this problem formulation:

Parameter	**Symbol**	**Value**
Observation dimension	n_o	125
Action dimension	n_a	29
Episode length	T	500
Maximum joint position	$q_i^{\max}$	π rad
Minimum joint position	$q_i^{\min}$	$-\pi$ rad
Goal sampling radius (min)	-	1.0 m
Goal sampling radius (max)	-	3.0 m
Goal tolerance	ϵ_g	0.2 m
Minimum CoM height	$h_{\min}$	0.3 m
Discount factor	γ	0.99
Control frequency	-	10 Hz
Proportional gain	K_p	20.0
Derivative gain	K_d	0.5
Hidden layer size	d_h	256
Velocity tracking weight	w_v	1.0
Distance reward weight	w_d	1.0
Alignment reward weight	w_a	0.5
Smoothness weight	w_s	0.01
Success bonus	R_{success}	5.0
Velocity tracking std	σ_v	0.2
Distance reward std	σ_d	2.0

Table 1: Summary of hyperparameters in the problem formulation.

7 Conclusion

This formal problem formulation defines a finite-horizon MDP for humanoid robot locomotion with clearly specified state and action spaces, reward structure, and learning objectives. The framework is designed to enable the training

of a control policy in simulation that can be subsequently validated through sim-to-sim transfer and eventually deployed on the real Unitree G1 robot.